arXiv:2312.08583v2 [cs.CL] 18 Dec 2023

ZeroQuant(4+2): Redefining LLMs Quantization with a New FP6-Centric Strategy for Diverse Generative Tasks

Xiaoxia Wu*, Haojun Xia†, Stephen Youn, Zhen Zheng, Shiyang Chen‡, Arash Bakhtiari, Michael Wyatt, Reza Yazdani Aminabadi, Yuxiong He, Olatunji Ruwase, Leon Song*, Zhewei Yao

DeepSpeed of Microsoft

Abstract

This study examines 4-bit quantization methods like GPTQ in large language models (LLMs), highlighting GPTQ's overfitting and limited enhancement in Zero-Shot tasks. While prior works merely focusing on zero-shot measurement, we extend task scope to more generative categories such as code generation and abstractive summarization, in which we found that INT4 quantization can significantly underperform. However, simply shifting to higher precision formats like FP6 has been particularly challenging, thus overlooked, due to poor performance caused by the lack of sophisticated integration and system acceleration strategies on current AI hardware. Our results show that FP6, even with a coarse-grain quantization scheme, performs robustly across various algorithms and tasks, demonstrating its superiority in accuracy and versatility. Notably, with the FP6 quantization, StarCoder-15B model performs comparably to its FP16 counterpart in code generation, and for smaller models like the 406M it closely matches their baselines in summarization. Neither can be achieved by INT4. To better accommodate various AI hardware and achieve the best system performance, we propose a novel 4+2 design for FP6 to achieve similar latency to the state-of-the-art INT4 fine-grain quantization. With our design, FP6 can become a promising solution to the current 4-bit quantization methods used in LLMs.[1]

1 Introduction

Large Language Models (LLMs) such as GPT-3 [5] have significantly advanced the field of natural language processing. These models have shown exceptional capabilities in various complex tasks, from text generation to language understanding. However, the widespread adoption of LLMs is challenged by their extensive computational and memory demands. This issue is particularly acute in resource-constrained environments, where deploying such large models is not feasible. To mitigate these challenges, post-training quantization has been recognized as a crucial technique [6, 20, 46, 41]. It enables the compression of these models for efficient utilization in limited-resource settings without the need for extensive retraining. Nevertheless, this approach often necessitates a balance between reducing the model size and maintaining accuracy [14].

Recent developments in the field of quantization, particularly in 4-bit quantization, have demonstrated potential in compressing LLMs effectively as their quality drops are greatly minimized due to advance algorithm design such as GPTQ [19] and LoRC [69]. However, these advancements have predominantly focused on zero-shot evaluations and the acceptable quality drops are for larger model size greater 13B, yet they often come with a significant trade-off for smaller model size such as 1B. Moreover, they only focus on zero-shot measurement [62, 69]. In production environments, where replicating the original model's

*Our corresponding authors are Xiaoxia Wu (xiaoxiawu@microsoft.com) and Leon Song (leonsong@microsoft.com).
†Haojun Xia is currently a PhD student at The University of Sydney, Australia.
‡Shiyang Chen is currently a PhD student at Rutgers University, USA.
[1]Code will be released soon as a part of https://github.com/microsoft/DeepSpeed

performance across different tasks is critical, any loss of model quality is a major concern. Existing methods, while innovative, do not fully address the practical requirements for deploying LLMs in real-world applications.

Contribution. To address these challenges, our contributions are as follows:

- **Broadened Evaluation Scope and Quantization Analysis.** Our study reveals that existing quantization methods like GPTQ tend to overfit to calibrated datasets. More significantly, we have broadened the scope of 4-bit quantization analysis in LLMs to include tasks beyond Zero-Shot, such as code generation and abstractive summarization. We discover that INT4 quantization often underperforms, especially in smaller models, even as large as 13 billion parameters, exemplified by LLaMA-13b. See Section 3 for details.

- **Superior Performance with FP6 Quantization.** We illustrate that FP6, employing a basic round-to-nearest (RTN) algorithm and a coarse-grain quantization approach, consistently achieves accuracy on par with full-precision models, proving highly effective across a broad spectrum of generative tasks. The StarCoder-13B model with FP6 quantization matches the performance of its FP16 equivalent in code generation tasks. For smaller models such as the 406M, it aligns closely with baseline results in summarization. These achievements are beyond the capabilities of INT4 quantization. For a more in-depth exploration, refer to Section 4.

- **Innovative 4+2 FP6 Design.** We introduce an innovative 4+2 design for FP6 that overcomes prior integration and acceleration issues on AI hardware. This design attains latency similar to the state-of-the-art INT4 fine-grain quantization, establishing FP6 as a viable alternative to existing 4-bit quantization methods in LLMs. See Section 5 for details.

2 Related Work

In this study, we specifically focus on the quantization of Large Language Models (LLMs), diverging from other neural network architectures like BERT and ResNet models, which have been extensively explored in existing literature [54, 71, 17, 63, 3, 16, 55, 28].

Quantization generally refers to employing low-precision weights and activations to leverage faster arithmetic cores, such as INT8/INT4 tensor cores [26]. However, the distinctive bandwidth constraints of LLMs have popularized weight-only quantization methods [71, 19, 68, 62, 61] as a strategy to reduce the memory footprint of these models.

Most previous research evaluates the impact of quantization using metrics like zero-shot perplexity or accuracy [66, 19, 8, 2, 29]. However, given that the main real-world applications of LLMs, such as ChatGPT [5] and Codex [21], revolve around generation-based tasks, a more comprehensive evaluation framework for quantized LLMs is warranted.

While many studies focus on integer data formats for their ease of simulation and extensive ecosystem support [31, 15, 19, 8, 29, 27], recent works have also demonstrated the effectiveness of floating-point formats [62, 13]. Nonetheless, these investigations typically center on conventional bit precisions such as 2/4/8 bits. Some research, like GPTQ, delves into 3-bit precision, but number concatenation methods, as discussed in Section 5, limit their system performance.

Finally, while the push for lower precision quantization continues, the practicality of deploying a model of size xB quantized to $2y$ bits over a $2xB$ model with y bits quantization is often overlooked, despite potential quality advantages [14, 69]. Our paper seeks to find an optimal balance where the quantized model retains similar accuracy to a full-precision model, an aspect largely missing in current literature [50, 69, 53, 70, 34, 23, 59, 22, 67].

3 Comprehensive Evaluation is Needed

For completeness, we here explain some foundational terminology and concepts in quantization.

Table 1: **Zero-Shot Evaluation (Perplexity↓)**. GPTQ quantization algorihtm for INT4 weight (W4A16) on LLaMA-1B (Left) and LLaMA-13B (Right). Different calibration datasets result in different results.

Precision	FGQ	Dataset for GPTQ	LLaMA-1B (4096-seq)				LLaMA-13B (2048-seq)			
			PTB	PTB-new	C4	C4-new	PTB	PTB-new	C4	C4-new
FP16	N/A	N/A	37.39	58.34	8.91	9.4	19.23	28.10	6.61	6.8
INT4-GPTQ	✗	PTB	**49.80**	**64.01**	10.00	10.49	**19.68**	**28.71**	6.91	7.17
	✗	C4	719.21	693.48	9.84	10.37	21.31	30.01	6.84	7.09
	✓	C4	1399.89	1396.76	**9.34**	**9.84**	22.14	29.83	**6.74**	**6.95**

Integer Quantization. Consider a full-precision $\mathbf{x} \in \mathbb{R}^d$ and its quantized counterpart $\mathbf{x}_q \in \mathbb{R}^d$. The mapping strategy from $\mathbf{x}$ to $\mathbf{x}_q$ is

$$\mathbf{x}_q = S \left\lceil clamp((\mathbf{x} - x_{\text{zero}}\mathbf{1})/S; 0, 2^{b-1} - 1) \right\rceil + x_{\text{zero}}\mathbf{1}, \tag{1}$$

where *clamp* the function map its input values to a given range from -2^{b-1} to $2^{b-1} - 1$. x_{zero} serves as a reference point, aiming to minimize any bias and can be defined as the minimum value, such as $x_{\text{zero}} = min(\mathbf{x})$. The parameter b represents the number of bits used to represent the quantized value. $\lceil \cdot \rceil$ is the rounding operator, and $S \in \mathbb{R}$ is the scaling factor. For example, S can be computed as the distance between the maximum and minimum of the absolute elements in the $\mathbf{x}$ vector, i.e., $S = max(\mathbf{x}) - min(\mathbf{x})$. For 4-bit integer (INT4) quantization on LLMs, this algorithm with x_{zero} strictly greater zero (called asymmetric quantization) has been widely adopted and proved to be much better than that by setting $x_{\text{zero}} = 0$ based on prior works [61, 62, 68, 69]. For history and details on how to set the parameters, see [20].

Fine-grain Quantization (FGQ) and Coarse-grain Quantization (CGQ) relates to the value of d in previous paragraph. Suppose a matrix of dimension $n \times n$ is vectorized into n^2 elements which are then grouped into blocks of size d, yielding n^2/d groups. When the block size d is set to 1, it equates to the matrix's original precision. FGQ comes into play when the block size is smaller than specific thresholds like 64, 128, or 256, given a matrix size of $n \geq 1024$. This method focuses on adjusting the precision within each row or column of the matrix at a finer level. CGQ, in contrast, uses a block size equivalent to the row size ($d \geq 1024$ in this case), resulting in a coarser quantization approach. FGQ gained significant attention in the realm of LLMs because the values in the weight matrices tend to have a wider distribution, as noted in [69].

Alongside FGQ or CGQ, specific algorithms are employed for precision mapping in quantization. Given the focus on 4-bit quantization and the demonstrated efficacy of the INT4 format over FP4 (as detailed in the appendix) [62], the investigation primarily centers on a straightforward method, RTN, and the increasingly recognized and impactful algorithm, GPTQ [18, 19], with a solid foundation background [33, 24]. We now explain them briefly below:

- **RTN. R**ound-**t**o-**n**earest neighborhood simply map the weight matrices to its low-precision counterpart based on Equation 1.
- **GPTQ. G**enerative **P**re-trained **T**ransformer **Q**uantization is a more advanced method of leveraging the activation information, which requires the inverse of the second-order input information. According to [18, 19], it reduces the precision of the model's weights to a lower bit representation (down to 3 or 4 bits per weight) without significant accuracy loss. Their code implementation is structured in a layer-by-layer manner, transferring the computational burden to the CPU when it's not in use. This strategy allows for the execution of massive models, like those with 175B parameters, on a single GPU, overcoming previous limitations of scale and complexity. GPTQ enhances the practical deployment of these models, particularly in memory and computationally constrained environments.

In addition to the algorithms previously mentioned, there has been significant progress in Post-Training Quantization (PTQ) for LLMs, highlighted by innovations such as SmoothQuant [66], AWQ [38], Quip [8], SqueezeLLM [29], QUIK [2], and LLM-FP4 [40] and many more [69, 14]. These methodologies, however, often necessitate the use of additional sparsity matrices or extra procedures to pinpoint sensitive weights.

Table 2: **Zero-Shot Evaluation (Perplexity↓).** Compare between GPTQ_{C4} and RTN quantization algorithms for INT4 weight (W4A16) on LLaMA of size 1B, 13B and 65B. We apply fine-grain quantization (FGQ) in which the block-size is 256 elements per scale (LLaMA-1B's block-size is 128). We also report results for coarse-grain quantization (CGQ) (per row per scale). The evaluation datasets are Wikitext2, PTB, C4, PTB-new, and C4-new.

Quant	Precision	LLaMA-1B (4096-seq)	LLaMA-13B (2048-seq)	LLaMA-65B (2048-seq)
	FP16	$24.31_{7.53/37.39/8.91/58.34/9.40}$	$13.16_{5.09/19.23/6.61/28.10/6.80}$	$6.41_{3.56/8.00/5.62/8.88/5.98}$
FGQ	INT4-GPTQ	$564.73_{7.83/1399.89/9.34/1396.76/9.84}$	$14.19_{5.28/22.14/6.74/29.83/6.95}$	$6.61_{3.81/8.17/5.73/9.20/6.13}$
	INT4-RTN	$22.76_{7.98/34.03/9.50/52.28/10.00}$	$14.32_{5.35/22.49/6.80/29.96/7.00}$	$7.30_{3.79/10.13/5.74/10.54/6.31}$
CGQ	INT4-GPTQ	$288.22_{8.20/719.21/9.84/693.48/10.37}$	$14.13_{5.37/21.31/6.84/30.01/7.09}$	$7.17_{4.12/10.50/5.83/9.16/6.21}$
	INT4-RTN	$34.29_{8.33/55.52/10.15/86.85/10.62}$	$14.32_{5.55/20.95/6.97/30.91/7.22}$	$7.72_{4.20/10.59/5.90/11.44/6.47}$

Furthermore, the majority of these studies concentrate predominantly on zero-shot perplexity and accuracy performance [69, 19, 62]. Yet, the extent to which these findings can be generalized to other generative tasks remains to be fully explored.

Experiment Settings We assess performance across three metrics: Zero-Shot tasks, Code Generation, and Summarization. We also perform try to implement comparative experiments for those chat-based models and judged by GPT-4 based on the FastChat codes [75]. Despite this, due to significant variability in our findings, we concluded that there is no clear link between bit precision and performance. These results are detailed further in the Appendix of our study.

- **Zero-Shot Tasks.** Leveraging open-source repositories[2], we applied GPTQ quantization algorithms to measure both perplexity and accuracy in zero-shot contexts. The datasets used for perplexity measurement include PTB [42], Wikitext [43], and C4 [51].[3] For accuracy, we randomly pick ten tasks: ARC (Challenge/Easy) [4], BoolQ [9], CB [12], Copa [1], PIQA [56], RTE [11], WSC [35], Storycloze [45]). Calibration for GPTQ used 128 (32) samples for LLaMa-1B/13B (-65B) models.[4]. We believe the results generalize to other models family such sh OPT [72] and BLOOM [52], The experiments were deterministic, using the seed 123.
- **Code Generation.** Following [76] and their open-source implementation[5], we adapted non-greedy generation settings (n=20, t=0.2, top_p=0.95). To mitigate variance, nine random seeds {111,222,..., 888, 1111} were employed. The models evaluated included CodeGeeX2-6B, StarCoder-15B [36], and CodeLLaMA-34B [39].[6] We focused on Python and JavaScript, noting instability in other programming languages.
- **Summarization Tasks.** Based on [37, 61] and their open-source codes,[7] we utilized BART-large, fine-tuned for CNNDailyMail [25] and XSum [47] summarization tasks.[8] Default settings were applied for all other parameters.

We focus on thes experiments with FP16 activation and INT4 weights on LLMs. Our experimental setup

[2] https://github.com/microsoft/DeepSpeed/tree/master/deepspeed/compression, https://github.com/qwopqwop200/GPTQ-for-LLaMa, and https://github.com/jerry-chee/QuIP

[3] Following the approach in gptq-for-llama, we added two new validation sets: PTB-new, using the PTB test dataset, and C4-new, comprising the first 256×seqlength. These new sets are implemented as per QuIP.

[4] LLaMA-13B/65B are from [57] and LLaMA-1B is from [65]. They can be downloaded from HuggingFace with names: 'princeton-nlp/Sheared-LLaMA-1.3B', 'huggyllama/llama-13b', 'huggyllama/llama-65b'.

[5] https://github.com/THUDM/CodeGeeX2

[6] Available as 'THUDM/codegeex2-6b', 'bigcode/starcoder', and 'codefuse-ai/CodeFuse-CodeLlama-34B' on HuggingFace.

[7] https://github.com/amazon-science/dq-bart

[8] Models available as 'facebook/bart-large-cnn' and 'facebook/bart-large-xsum' on HuggingFace.

Table 3: **Zero-Shot Evaluation (Accuracy↑)**. Compare between $GPTQ_{C4}$ and RTN quantization algorithms for INT4 weight (W4A16) on LLaMA-1B (Top) and LLaMA-13B (Bottom). We apply fine-grain quantization (FGQ) in which the block-size is 256 elements per scale except for LLaMA-1B's (which is 128). arcC (arcE) stands for arc_challenges (arc_easy).

Models	Precision (FGQ)	arcC	arcE	boolq	cb	copa	piqa	rte	wic	wsc	storycloze	MEAN
LLaMA-1B (4096-seq)	FP16	26.71	53.11	61.13	39.29	76.00	73.83	50.18	50.00	36.54	69.64	53.64
	INT4-GPTQ	**26.37**	50.59	**61.59**	46.43	**79.00**	**73.34**	48.01	50.00	36.54	68.24	**54.01**
	INT4-RTN	26.11	**51.09**	58.07	**50.00**	74.00	72.91	**48.38**	50.00	36.54	**68.36**	53.55
LLaMA-13B (2048-seq)	FP16	43.86	74.58	68.53	50.00	90.00	79.00	65.34	50.00	35.58	78.23	63.51
	INT4-GPTQ	43.00	73.44	**67.83**	41.07	**93.00**	78.78	62.45	**50.16**	36.54	78.17	62.44
	INT4-RTN	**44.03**	**74.45**	67.37	**44.64**	91.00	**78.84**	**63.18**	49.84	36.54	**78.42**	**62.83**
LLaMA-65B (2048-seq)	FP16	47.01	75.08	82.32	64.29	91.00	81.61	71.48	58.31	60.58	79.57	71.13
	INT4-GPTQ	46.84	75.08	80.76	58.93	94.00	81.18	**72.92**	56.27	60.58	79.31	70.59
	INT4-RTN	**47.10**	**75.25**	**81.47**	**62.50**	**95.00**	**81.23**	69.68	**57.21**	**62.50**	**79.63**	**71.16**

Table 4: **Generation Tasks (Rouge↑ or Pass@1↑)**. INT4 weight (W4A16) quantization on $BART_{406M}$, CodeGeeX2-6B, StarCoder-15B, and CodeLLaMA-34B models (left to right) Using RTN. Note that summarization tasks use two separate $BART_{406M}$ versions fine-tuned by CNN/XSUM and code generation tasks in Human-X including Python and JavaScript (as the variances of other tasks such as CPP, Go and RUST are higher and so not included), averaged over at least 8 repeated experiments with standard deviation.

Precision	$BART_{406M}$(R1/R2/RL) XSUM (CNN)	CodeGeeX2-6B (pass@1) Python	CodeGeeX2-6B (pass@1) Java-Scrpit	StarCoder-15B (pass@1) Python	StarCoder-15B (pass@1) Java-Scrpit	CodeLLaMA-34B (pass@1) Python	CodeLLaMA-34B (pass@1) Java-Scrpit
FP16	45.49/22.39/37.28 (44.07/21.09/30.65)	34.04±1.70	31.50±2.62	35.43±2.21	33.67±2.02	43.22±2.21	45.05±1.60
INT4 (CGQ)	43.76/20.76/35.82 (43.67/20.68/30.07)	33.08±2.07	25.15±1.97	33.20±1.40	32.18±1.29	39.84±1.64	43.45±2.05
INT4 (FGQ)	44.82/21.63/36.48 (43.73/20.72/30.24)	29.80±1.76	28.35±2.36	35.64±2.52	32.32±2.01	46.88±1.87	43.22±1.36

includes a single-GPU environment, utilizing either a V100-32g or H100-80g GPU. Based on the results Table 1, Table 2, and Table 3, we make the following two key observations.

GPTQ's Tendency to Overfit. Although GPTQ is innovative in post-training quantization, it tends to overfit to particular datasets, especially noticeable in its fine-grain quantization results. As indicated in Table 1, we see that if we calibrate with specific dataset such as C4 for GPTQ, then the performance for this C4 dataset would be much better (see 9.34 or 6.74 using FGQ), while other datasets such as PTB would result in much worse performance (see 1399.89 and 22.14 using FGQ). Independently, [60] also notices this issue while examining LLaMA-7B.

It is admitted that the over-fitting phenomena is less severe for larger models (moving from 1B to 13B or 65B). Indeed, as shown in Table 2, we see that LLaMA-65B using GPTQ on FGQ for INT4-weight results in the best average perplexity 6.61 comparing to RTN (7.17), much closer to the baseline 6.41. However, its effectiveness in enhancing Zero-Shot performance is somewhat limited (detailed in Table 3), suggesting a gap in its adaptability across various language modeling scenarios and highlighting the need for robustness in model evaluation. In particularly, we presents in Table 3 the comparison between *RTN* and *GPTQ* on INT4 weight while keep the activation untouched, we can not claim that *GPTQ* and *RTN* are better than another based on zero-shot performance. In fact, for LLaMA-65B, the performance for *RTN* is surprisingly better than the one of FP16.

Expanding Evaluation Methods for Generative Models. Our current analysis, mainly centered on zero-shot performance as shown in Table 2 and Table 3, highlights the need for a broader scope in evaluation techniques. The core strength of LLMs lies in their ability to generate sequences. Therefore, this paper

focuses on assessing summarization and code generation, as elaborated in Table 4.[9] This strategy underlines the importance of comprehensive and detailed testing methods that extend beyond zero-shot learning, aiming to fully evaluate the generative capabilities of LLMs. The data in Table 4 show a notable difference in performance with INT4, especially when compared to standard benchmarks. For example, the performance of the CodeLLaMA-34B model in Java-Script drops from 45.05 (FP16) to 43.45 (INT4, CGQ) or 43.22 (INT4, FGQ), a decrease of 1.6 and 1.83 points, respectively. While FGQ on INT4 offers considerable improvements over CGQ, gaps compared to FP16 persist, particularly for smaller models and in Java Scripts. Interestingly, the INT4 CodeLLaMA-34B on FGQ achieves 46.88 in Python code, surpassing its baseline, whereas the INT4 CodeGeeX2-6B on FGQ scores only 29.8, falling behind even its INT4-CGQ performance. This highlights the inconsistency of INT4.

These results emphasize the need for research into the effectiveness of INT4 in complex generative tasks.

4 Sweet Spot Solution: FP6

Building on previous discussions around the challenges and limitations associated with INT4 quantization, particularly its instability and subpar outcomes in code generation and summarization tasks, this section delves into an emerging area of interest in floating point quantization research. Recent studies have increasingly focused on the use of floating point quantization for handling weights or activations within LLMs [62, 40, 74, 44, 7, 32, 58]. Notably, a simple FP8's application in activation processes has shown remarkable improvements over the use of INT8 [62]. Inspired by these advancements, a critical question arises: *Could increasing the bit precision, for instance to 5 or 6 bits, offer more stable and robust outcomes in generative tasks? This section aims to explore the extent of FP6's (FP5's) effectiveness and its resilience to different quantization algorithms, offering a potential solution to the dilemma posed by previous INT4 quantization challenges.*

For completeness, we provide a simplified overview of the floating-point format. For a detailed explanation, please refer to [10]. A standard floating point number comprises three parts: the sign bit, the exponent bits, and the mantissa bits. This can be simplified as:

$$x = S \times 2^{E-b} \times M, \tag{2}$$

where S represents the sign (± 1), E denotes the exponent ($[0, 2^e - 1]$, with e being the count of exponent bits), b is the bias for the exponent (usually $2^{e-1} - 1$), and M signifies the mantissa ($[0, 2)$).

Following the implementation of [73], the maximum/minimum achievable value in FP6_{E3M2} is ± 28 ($\pm 1 \times 2^4 \times 1.75$).[10] The FP16 (or BF16) weight matrix undergoes quantization as follows:

$$\hat{W}_{fp16} \approx Quant(W_{fp16}) = S_{fp16} \times W_{fp6}, \tag{3}$$

where W_{fp16} is the original full precision weight, $Quant(\cdot)$ symbolizes the quantization function, S_{fp16} is the scaling factor, and W_{fp6} is the FP6_{E3M2} number. The scaling factor S_{fp16} is computed by: $S_{fp16} = max(abs(W_{fp16}))/28$, thereby ensuring optimal use of W_{fp16}'s range without compromising on precision. Please see Section 5 for additional customizations in FP6_{E3M2}. Similar settting is defined for F5_{E3M1}.

Why not INT6 instead of FP6. The choice of FP6 over INT6 is driven by two key factors: firstly, the FP format simplifies conversion processes, as final computations are typically performed using FP16 or BF16. Secondly, there is no observed difference in accuracy between these formats, as supported by findings in [62] , eliminating the need for additional experimental validation.

[9] It should be noted that GPTQ tends to overfit to the calibrated dataset and poses implementation challenges, leading us to solely utilize RTN for our evaluations.

[10] Note that for some other work such as [44] has specific configuration defined for exceptional values like NaN and $\pm\infty$. However, these are not included in our weight quantization process using FP6_{E3M2}. We do not think this slight difference will greatly impact the model performance.

4.1 Results of FP6 and FP5 on all tasks

Table 5: **Generation Tasks (Rouge or Pass@1 ↑)**. Comparative results of quantizations on $BART_{406M}$, CodeGeeX2-6B, StarCoder-15B, and CodeLLaMA-34B models (left to right). Summarization tasks using two $BART_{406M}$ versions fine-tuned by CNN/XSUM and code generation tasks in Python and JavaScript, averaged over 8 iterations with standard deviation. FP6 (FP5) format is E3M2 (E3M1).

Precision (RTN)	$BART_{406M}$(R1/R2/RL) XSUM (CNN)	CodeGeeX2-6B (pass@1) Python	CodeGeeX2-6B (pass@1) Java-Scrpit	StarCoder-15B (pass@1) Python	StarCoder-15B (pass@1) Java-Scrpit	CodeLLaMA-34B (pass@1) Python	CodeLLaMA-34B (pass@1) Java-Scrpit
FP16	45.49/22.39/37.28 (44.07/21.09/30.65)	34.04±1.70	31.50±2.62	35.43±2.21	33.67±2.02	43.22±2.21	45.05±1.60
INT4 (CGQ)	43.76/20.76/35.82 (43.67/20.68/30.07)	33.08±2.07	25.15±1.97	33.20±1.40	32.18±1.29	39.84±1.64	43.45±2.05
INT4 (FGQ)	44.82/21.63/36.48 (43.73/20.72/30.24)	29.80±1.76	28.35±2.36	35.64±2.52	32.32±2.01	46.88±1.87	43.22±1.36
FP5 (CGQ)	45.12/22.11/36.96 (44.13/21.19/30.68)	30.56±2.12	28.43±2.39	34.30±1.29	33.00±1.54	40.55±0.92	43.29±1.93
FP6 (CGQ)	45.37/22.20/37.11 (44.06/21.07/30.67)	34.10±2.09	31.61±1.74	35.64±1.26	33.60±1.91	44.31±1.88	44.51±1.30

Table 6: **Zero-Shot Evaluation (Perplexity↓).** Compare between $GPTQ_{C4}$ and RTN quantization algorithms for INT4 weight (W4A16) on LLaMA of size 1B, 13B and 65B. We apply fine-grain quantization (FGQ) in which the block-size is 256 elements per scale (LLaMA-1B's block-size is 128). We also report results for coarse-grain quantization (CGQ) (per row per scale). The evaluation datasets are Wikitext2, PTB, C4, PTB-new, and C4-new. FP6 (FP5) format is E3M2 (E3M1)

Precision	FGQ	LLaMA-1B (4096-seq)	LLaMA-13B (2048-seq)	LLaMA-65B (2048-seq)
FP16	N/A	$24.31_{7.53/37.39/8.91/58.34/9.40}$	$13.16_{5.09/19.23/6.61/28.10/6.80}$	$6.41_{3.56/8.00/5.62/8.88/5.98}$
INT4-$GPTQ_{C4}$	✓	$564.73_{7.83/1399.89/9.34/1396.76/9.84}$	$14.19_{5.28/22.14/6.74/29.83/6.95}$	$6.61_{3.81/8.17/5.73/9.20/6.13}$
INT4-RTN	✓	$22.76_{7.98/34.03/9.50/52.28/10.00}$	$14.32_{5.35/22.49/6.80/29.96/7.00}$	$7.30_{3.79/10.13/5.74/10.54/6.31}$
INT4-$GPTQ_{C4}$	✗	$288.22_{8.20/719.21/9.84/693.48/10.37}$	$14.13_{5.37/21.31/6.84/30.01/7.09}$	$7.17_{4.12/10.50/5.83/9.16/6.21}$
INT4-RTN	✗	$34.29_{8.33/55.52/10.15/86.85/10.62}$	$14.32_{5.55/20.95/6.97/30.91/7.22}$	$7.72_{4.20/10.59/5.90/11.44/6.47}$
FP5-$GPTQ_{C4}$	✓	$44.29_{7.74/72.63/9.19/122.20/9.68}$	$13.76_{5.22/20.43/6.71/29.53/6.92}$	$6.50_{3.67/8.15/5.68/8.95/6.04}$
FP5-RTN	✓	$28.52_{7.78/44.09/9.23/71.79/9.71}$	$13.95_{5.20/21.01/6.71/29.92/6.92}$	$6.83_{3.70/9.73/5.69/8.98/6.06}$
FP5-$GPTQ_{C4}$	✗	$32.03_{7.77/50.66/9.23/82.73/9.76}$	$13.90_{5.22/21.02/6.71/29.64/6.93}$	$6.46_{3.68/7.83/5.70/9.13/5.98}$
FP5-RTN	✗	$27.92_{7.83/41.80/9.27/70.94/9.77}$	$14.10_{5.22/21.52/6.72/30.13/6.93}$	$6.50_{3.68/8.08/5.69/8.98/6.06}$
FP6-$GPTQ_{C4}$	✓	$22.77_{7.59/34.04/8.98/53.76/9.47}$	$13.36_{5.13/19.67/6.63/28.53/6.83}$	$6.47_{3.59/8.12/5.63/8.91/6.10}$
FP6-RTN	✓	$23.42_{7.60/35.84/8.99/55.20/9.49}$	$13.24_{5.12/19.43/6.63/28.19/6.83}$	$6.44_{3.58/8.04/5.63/8.92/6.05}$
FP6-$GPTQ_{C4}$	✗	$23.58_{7.59/35.76/8.98/56.10/9.47}$	$13.23_{5.12/19.34/6.64/28.20/6.83}$	$6.42_{3.61/8.01/5.63/8.87/6.00}$
FP6-RTN	✗	$24.83_{7.60/38.79/8.99/59.26/9.49}$	$13.09_{5.12/19.06/6.64/27.81/6.83}$	$6.42_{3.59/8.01/5.63/8.89/5.99}$

We conduct the experiments described in Section 3, namely code generation, summarization and zero-shot experiments. The results are shown in Table 5 and Table 6. Additional results are presented in Table A.1 and Table A.2, which we defer to appendix as it we do not find a clear link between bit precision and performance.

In general, the FP6 quantization method, particularly with CGQ, stands out in this analysis, offering a balance of high performance and robustness across different tasks, models and even quantiztion algorithms (RTN and GPTQ), a notable improvement over both FP5 and INT4 quantizations. In addition, we make the following observation:

FP5 Performance. FP5 with CGQ shows an improvement over INT4 quantization but still does not reach the high performance levels of FP16. The gap between FP5 and its baseline is particularly noticeable in the Python and JavaScript code generation tasks across CodeGeeX2-6B, StarCoder-15B, and CodeLLaMA-34B.

FP6 Robustness. FP6 quantization, especially with CGQ, demonstrates a significant advancement, nearly matching the FP16 baseline across various tasks and models. This quantization method not only narrows the performance gap seen in FP5 and INT4 but also shows robustness in handling different tasks. The robustness is further accentuated when comparing CGQ and FGQ within the FP6 framework (as there

is little difference between CGQ and FGQ), where FP6 with CGQ consistently maintains high performance close to baseline, indicating its effectiveness and stability across different scenarios. Moreover, FP6 is robust to quantization algorithms: either RTN or GPTQ results in similar results, particularly for CodeLLaMA-34B, as shwon in Table 6.

5 System Support Discussion

5.1 4+2 format for FP6

In addressing the challenges of utilizing a non-standard 6-bit number format, which deviates from the conventional power-of-2 numerical systems (termed as "odd bit precision setting"), we propose a novel approach. This method is distinct from the two commonly considered strategies:

1. The first approach involves directly converting the 6-bit format into an 8-bit floating-point (FP8) format. While this is a straightforward solution, it unfortunately negates the primary benefit of the 6-bit format, which is to conserve memory.

2. The second approach entails grouping several 6-bit numbers together in a continuous memory block and representing them using either 32-bit integers (INT32) or a 32-bit floating-point (FP32) format. This method maintains the memory-saving advantage but adds complexity to the dequantization process.

Our unique strategy, however, focuses on dividing the 6-bit number into two distinct sub-numbers: the first sub-number representing the initial 4 bits, and the second sub-number accounting for the remaining 2 bits. Our poposed "4+2" method can be seen as an advanced variation of the second standard approach. The 4+2 bit division is based on the fundamental principle that any positive integer can be expressed as a sum of powers of 2. With this foundation, we divide the 6-bit number into two components:

- The first part, comprising the initial 4 bits, handles elements such as the sign bit and a 3-bit exponent.

- The second part, containing the remaining 2 bits, is dedicated to the 2-bit mantissa.

This division into 4+2 bits facilitates simultaneous loading and dequantization of these sub-numbers, culminating in the generation of the final 16-bit floating-point (FP16) weight. Our approach innovatively balances the need for reduced memory footprint with the practicalities of dequantization, particularly in addressing the challenges of memory access across segmented numbers.

5.2 Bias Shift

Dequantizing FP6 to FP16 during runtime on GPUs can be significantly resource-intensive, primarily due to the complexity involved in manipulating the exponent field, as detailed in Section 4.

The bias term for the exponent, typically determined by the exponent bits, is 3 for FP6 and 15 for FP16. Mathematically, the process of dequantizing FP6 to FP16 (excluding the sign) is represented as:

$$2^{E_{\text{FP16}}-15} \times M_{\text{FP16}} = 2^{E_{\text{FP6}}-3} \times M_{\text{FP6}}, \tag{4}$$

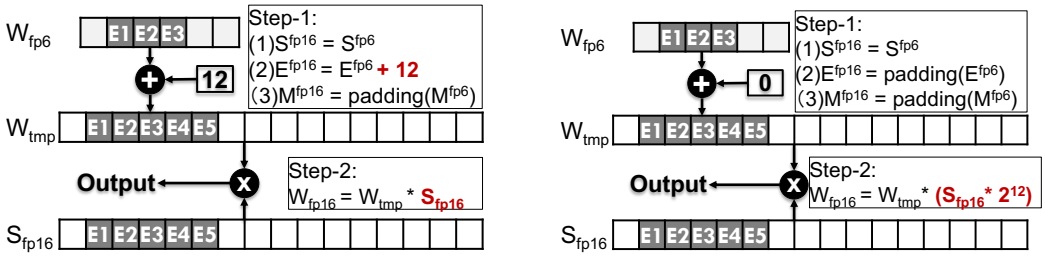
where the superscripts `FP16/FP6` indicate the respective format. It is noteworthy that the scaling factor dequantization can be done after matrix multiplication before the accumulation for fine-grained (sub-row) quantization schemes or after accumulation for coarse-grained (row-wise) quantization schemes.

While padding can easily adjust the mantissa, aligning the exponents requires more effort due to the difference in biases. An analogy can be drawn with converting an INT4 number back to a symmetric INT8 format: if INT4 employs a symmetric format (for the mantissa), zero padding suffices. However, in an asymmetric format, padding alone is inadequate, and additional steps are necessary.

To address this, we have customized our FP6 format with a non-standard exponent bias of 15. This modification does not affect precision or accuracy because:

$$S_{\text{FP16}} \times 2^{E-3} \times M = (S_{\text{FP16}} \times 2^{12}) \times 2^{E-15} \times M, \tag{5}$$

meaning the bias shift can be seamlessly integrated into the scaling factor. Crucially, since S_{FP16} is less than 1, multiplying it with 2^{12} still allows for accurate representation in FP16 format through simple exponent bit shifting, avoiding numerical errors.



(a) Before Bias Shift.

(b) After Bias Shift.

Figure 1: Comparison of the Traditional Method (Left) versus Our Proposed Method (Right): Demonstrating the Significant Run-time Advantages of Bias Shift.

Our bias shift method greatly simplifies the FP6-FP16 de-quantization process during runtime. To demonstrate this, we provide a side-by-side comparison in Figures 1a and 1b.

Figure 1a outlines the original two-step process for de-quantizing each FP6 weight. The first step involves casting W_{fp6} to W_{tmp}, and the second step requires multiplying by the quantization scale S_{fp16}. The most demanding part of this procedure is recalculating the exponent for W_{tmp}, which involves extracting E_{fp6} from W_{fp6}, adding 12, and then incorporating this back into W_{tmp}. Additionally, the process to de-quantize subnormal FP6 numbers[11] adds further complexity to the de-quantization during runtime.

However, with our bias shift strategy, as illustrated in Figure 1b, the exponent adjustment becomes a straightforward bit-level padding process. The addition of the constant integer 12, initially required in Step 1, can now be deferred to Step 2, **eliminating any runtime overhead**. This is possible because the multiplication of the quantization scales with the constant integer can be performed statically after the model is quantized and before runtime. Moreover, this streamlined approach also efficiently accommodates the de-quantization of subnormal numbers.

5.3 System evaluation

We conducted an evaluation of the latest GPU kernels across various weight-only quantization techniques, focusing on their system performance. This kernel-level assessment was carried out on the NVIDIA A100-40GB platform, running Linux 5.3.18 and CUDA 11.8. Our primary focus was on the performance of feed-forward (FFN) layers within the LLaMA models, specifically during the token generation phase, as detailed in [57]. Comprehensive data on matrix shapes and kernel latency is available in Appendix B. We employed cuBLAS [48] as our benchmark for non-quantized performance (W16A16). We also included cutting-edge kernel support for F INT4 FGQ quantization (W4A16) from TensorRT-LLM [49] for comparative analysis.[12]

[11]Subnormal numbers in floating-point formats are very small values, including zero, characterized by an exponent field of all zeros. When de-quantizing subnormal FP6 numbers, the new exponent should remain at zero, not adding 12, to maintain its subnormal status. Subsequently, FP16 is multiplied by 1.0×2^{12}.

[12]Between the supported block-size: 64 and 128, we chose 128.

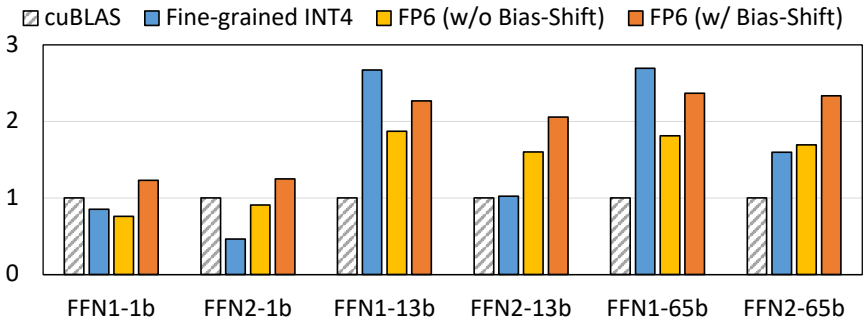


Figure 2: GPU Kernel Speedups compared to cuBLAS. FFN1 and FFN2 are defined for matrices of $4H \times H$ and $H \times 4H$ size, where H are defined by the size of the LLaMA models. LLaMA-1B, 13B and 65B are respectively 5504, 13824 and 22016. See detailed number in Table B.1

To elucidate the advantages of our proposed *Bias Shift* technique, detailed in Section 5.2, we also developed and tested an FP6 GPU kernel without *Bias Shift*.

According to the results presented in Figure 2, our FP6 kernel, enhanced with Bias-Shift, achieved speeds up to 2.37× faster and on average 1.92× faster than cuBLAS. Given that LLM inference is commonly constrained by limited GPU DRAM [64, 30], our approach effectively alleviates this bottleneck by minimizing model weight memory access. Moreover, our FP6 kernel outperforms the state-of-the-art fine-grained INT4 implementation in speed, being 1.06× faster on average for the first feed-forward layers (FFN1) and 2.05× faster for the second feed-forward layers (FFN2). It should be noted that Figure 2 provides only a snapshot of key outcomes, with a comprehensive system implementation and detailed performance analysis reserved for future work. Importantly, our FP6 kernel with Bias-Shift is on average 1.36× faster compared to the same FP6 kernel without *Bias Shift*, underscoring the crucial role of *Bias-Shift* as discussed in Section 5.2.

6 Discussion and Conclusion

Our paper introduces a novel approach to GPU kernel optimization, specifically targeting weight-only quantization methods. Despite the significance of our findings, they pave the way for further research and development in several areas.

Evaluation Scope. A primary limitation of our study is still the narrow scope of evaluation although we have span to code generation and summarization. We suggest a vast field for future research. Expanding the scope to include diverse tasks and a focus on both performance and accuracy could enhance the robustness of our methods.

Comparative Analysis. While our system evaluation provides valuable insights, it lacks in-depth comparison with state-of-the-art (SOTA) frameworks. A more comprehensive benchmarking against advanced frameworks would offer a clearer perspective on our approach's efficacy and areas for improvement.

Technique Adaptability. A notable aspect of our work is the adaptability of our techniques, particularly in bit-precision and bias-shift. The potential to adapt our methods to emerging standards, such as 5-bit quantization, demonstrates their flexibility and future applicability in various contexts.

Future Directions. The advancement of Post-Training Quantization (PTQ) methods and their integration with other techniques presents exciting future possibilities. Our research lays the foundation for further advancements in model optimization, such as the 3-bit precision from evolving quantization techniques.

In conclusion, our study marks a significant contribution to odd-bit GPU kernel optimization. It also opens avenues for broader research, exploring the full potential of model optimization and quantization across diverse applications.

Contributions

Xiaoxia Wu led model quality measurement and paper writing. Haojun Xia, contributing to system section writing, designed the FP6 acceleration strategy and system support. Stephen Youn, Zhen Zheng, and Shiyang Chen handled DeepSpeed integration. Arash Bakhtiari and Michael Wyatt managed framework testing and external integration. Reza Yazdani Aminabadi, Yuxiong He and Olatunji Ruwase provided key discussions. Leon Song is the overall leader for algorithm and system design. Zhewei Yao initiated the project and led the FP6/5 format design.

References

[1] Ardavan Afshar, Ioakeim Perros, Evangelos E Papalexakis, Elizabeth Searles, Joyce Ho, and Jimeng Sun. Copa: Constrained parafac2 for sparse & large datasets. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, pages 793–802, 2018.

[2] Saleh Ashkboos, Ilia Markov, Elias Frantar, Tingxuan Zhong, Xincheng Wang, Jie Ren, Torsten Hoefler, and Dan Alistarh. Towards end-to-end 4-bit inference on generative large language models. *arXiv preprint arXiv:2310.09259*, 2023.

[3] Haoli Bai, Wei Zhang, Lu Hou, Lifeng Shang, Jing Jin, Xin Jiang, Qun Liu, Michael Lyu, and Irwin King. Binarybert: Pushing the limit of bert quantization. *arXiv preprint arXiv:2012.15701*, 2020.

[4] Michael Boratko, Harshit Padigela, Divyendra Mikkilineni, Pritish Yuvraj, Rajarshi Das, Andrew McCallum, Maria Chang, Achille Fokoue-Nkoutche, Pavan Kapanipathi, Nicholas Mattei, et al. A systematic classification of knowledge, reasoning, and context within the arc dataset. *arXiv preprint arXiv:1806.00358*, 2018.

[5] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

[6] Yaohui Cai, Zhewei Yao, Zhen Dong, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Zeroq: A novel zero shot quantization framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13169–13178, 2020.

[7] Léopold Cambier, Anahita Bhiwandiwalla, Ting Gong, Mehran Nekuii, Oguz H Elibol, and Hanlin Tang. Shifted and squeezed 8-bit floating point format for low-precision training of deep neural networks. *arXiv preprint arXiv:2001.05674*, 2020.

[8] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. Quip: 2-bit quantization of large language models with guarantees. *arXiv preprint arXiv:2307.13304*, 2023.

[9] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*, 2019.

[10] Wikipedia contributors. Floating-point arithmetic — Wikipedia, the free encyclopedia, last edited 2023. [Online; accessed 8-December-2023].

[11] Ido Dagan, Dan Roth, Mark Sammons, and Fabio Massimo Zanzotto. Recognizing textual entailment: Models and applications. *Synthesis Lectures on Human Language Technologies*, 6(4):1–220, 2013.

[12] Marie-Catherine De Marneffe, Mandy Simons, and Judith Tonhauser. The commitmentbank: Investigating projection in naturally occurring discourse. In *proceedings of Sinn und Bedeutung*, volume 23, pages 107–124, 2019.

[13] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.

[14] Tim Dettmers and Luke Zettlemoyer. The case for 4-bit precision: k-bit inference scaling laws. *arXiv preprint arXiv:2212.09720*, 2022.

[15] Zhen Dong, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. HAWQ: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 293–302, 2019.

[16] Steven K Esser, Jeffrey L McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S Modha. Learned step size quantization. *arXiv preprint arXiv:1902.08153*, 2019.

[17] Angela Fan, Pierre Stock, Benjamin Graham, Edouard Grave, Remi Gribonval, Herve Jegou, and Armand Joulin. Training with quantization noise for extreme fixed-point compression. *arXiv preprint arXiv:2004.07320*, 2020.

[18] Elias Frantar and Dan Alistarh. Optimal brain compression: A framework for accurate post-training quantization and pruning. *arXiv preprint arXiv:2208.11580*, 2022.

[19] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[20] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. *arXiv preprint arXiv:2103.13630*, 2021.

[21] GitHub. Github copilot. https://github.com/features/copilot/, 2021.

[22] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–15, 2023.

[23] Han Guo, Philip Greengard, Eric P Xing, and Yoon Kim. Lq-lora: Low-rank plus quantized matrix decomposition for efficient language model finetuning. *arXiv preprint arXiv:2311.12023*, 2023.

[24] Babak Hassibi and David G Stork. Second order derivatives for network pruning: Optimal brain surgeon. In *Advances in neural information processing systems*, pages 164–171, 1993.

[25] Karl Moritz Hermann, Tomas Kocisky, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. Teaching machines to read and comprehend. *arXiv preprint arXiv:1506.03340*, 2015.

[26] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *The Journal of Machine Learning Research*, 18(1):6869–6898, 2017.

[27] Jeonghoon Kim, Jung Hyun Lee, Sungdong Kim, Joonsuk Park, Kang Min Yoo, Se Jung Kwon, and Dongsoo Lee. Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization. *arXiv preprint arXiv:2305.14152*, 2023.

[28] Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. I-bert: Integer-only bert quantization. In *International conference on machine learning*, pages 5506–5518. PMLR, 2021.

[29] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629*, 2023.

[30] Sehoon Kim, Coleman Hooper, Thanakul Wattanawong, Minwoo Kang, Ruohan Yan, Hasan Genc, Grace Dinh, Qijing Huang, Kurt Keutzer, Michael W Mahoney, et al. Full stack optimization of transformer inference: a survey. *arXiv preprint arXiv:2302.14017*, 2023.

[31] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.

[32] Andrey Kuzmin, Mart Van Baalen, Yuwei Ren, Markus Nagel, Jorn Peters, and Tijmen Blankevoort. Fp8 quantization: The power of the exponent. *arXiv preprint arXiv:2208.09225*, 2022.

[33] Yann LeCun, John S Denker, and Sara A Solla. Optimal brain damage. In *Advances in neural information processing systems*, pages 598–605, 1990.

[34] Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. Owq: Lessons learned from activation outliers for weight quantization in large language models. *arXiv preprint arXiv:2306.02272*, 2023.

[35] Hector Levesque, Ernest Davis, and Leora Morgenstern. The winograd schema challenge. In *Thirteenth International Conference on the Principles of Knowledge Representation and Reasoning*. Citeseer, 2012.

[36] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. Starcoder: may the source be with you! *2305.06161*, 2023.

[37] Zheng Li, Zijian Wang, Ming Tan, Ramesh Nallapati, Parminder Bhatia, Andrew Arnold, Bing Xiang, and Dan Roth. Dq-bart: Efficient sequence-to-sequence model via joint distillation and quantization. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 203–211, 2022.

[38] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

[39] Bingchang Liu, Chaoyu Chen, Cong Liao, Zi Gong, Huan Wang, Zhichao Lei, Ming Liang, Dajun Chen, Min Shen, Hailian Zhou, Hang Yu, and Jianguo Li. Mftcoder: Boosting code llms with multitask fine-tuning. *arXiv preprint arXiv*, 2023.

[40] Shih-yang Liu, Zechun Liu, Xijie Huang, Pingcheng Dong, and Kwang-Ting Cheng. Llm-fp4: 4-bit floating-point quantized transformers. *arXiv preprint arXiv:2310.16836*, 2023.

[41] Zhenhua Liu, Yunhe Wang, Kai Han, Wei Zhang, Siwei Ma, and Wen Gao. Post-training quantization for vision transformer. *Advances in Neural Information Processing Systems*, 34, 2021.

[42] Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Using Large Corpora*, page 273, 1994.

[43] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

[44] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. Fp8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.

[45] Nasrin Mostafazadeh, Michael Roth, Annie Louis, Nathanael Chambers, and James Allen. Lsdsem 2017 shared task: The story cloze test. In *Proceedings of the 2nd Workshop on Linking Models of Lexical, Sentential and Discourse-level Semantics*, pages 46–51, 2017.

[46] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or down? adaptive rounding for post-training quantization. In *International Conference on Machine Learning*, pages 7197–7206. PMLR, 2020.

[47] Sameer Narayan, Andre Martins, Alessandro Sordoni, Philip Bachman, Aaron Courville, and Yoshua Bengio. Don't give me the details, just the summary!: topic-aware convolutional neural networks for extreme summarization. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3706–3716, 2018.

[48] NVIDIA. cublas. "https://developer.nvidia.com/cublas", 2023.

[49] NVIDIA. Tensorrt-llm. "https://github.com/NVIDIA/TensorRT-LLM/", 2023.

[50] Gunho Park, Baeseong Park, Se Jung Kwon, Byeongwook Kim, Youngjoo Lee, and Dongsoo Lee. nuqmm: Quantized matmul for efficient inference of large-scale generative language models. *arXiv preprint arXiv:2206.09557*, 2022.

[51] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.

[52] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.

[53] Yuzhang Shang, Zhihang Yuan, Qiang Wu, and Zhen Dong. Pb-llm: Partially binarized large language models. *arXiv preprint arXiv:2310.00034*, 2023.

[54] Sheng Shen, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Q-BERT: Hessian based ultra low precision quantization of bert. In *AAAI*, pages 8815–8821, 2020.

[55] Chaofan Tao, Lu Hou, Wei Zhang, Lifeng Shang, Xin Jiang, Qun Liu, Ping Luo, and Ngai Wong. Compression of generative pre-trained language models via quantization. *arXiv preprint arXiv:2203.10705*, 2022.

[56] Sandeep Tata and Jignesh M Patel. Piqa: An algebra for querying protein data sets. In *15th International Conference on Scientific and Statistical Database Management, 2003.*, pages 141–150. IEEE, 2003.

[57] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[58] Mart van Baalen, Andrey Kuzmin, Suparna S Nair, Yuwei Ren, Eric Mahurin, Chirag Patel, Sundar Subramanian, Sanghyuk Lee, Markus Nagel, Joseph Soriaga, et al. Fp8 versus int8 for efficient deep learning inference. *arXiv preprint arXiv:2303.17951*, 2023.

[59] Xiuying Wei, Yunchen Zhang, Yuhang Li, Xiangguo Zhang, Ruihao Gong, Jinyang Guo, and Xianglong Liu. Outlier suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling. *arXiv preprint arXiv:2304.09145*, 2023.

[60] Miles Williams and Nikolaos Aletras. How does calibration data affect the post-training pruning and quantization of large language models? *arXiv preprint arXiv:2311.09755*, 2023.

[61] Xiaoxia Wu, Cheng Li, Reza Yazdani Aminabadi, Zhewei Yao, and Yuxiong He. Understanding int4 quantization for transformer models: Latency speedup, composability, and failure cases. *arXiv preprint arXiv:2301.12017*, 2023.

[62] Xiaoxia Wu, Zhewei Yao, and Yuxiong He. Zeroquant-fp: A leap forward in llms post-training w4a8 quantization using floating-point formats. *arXiv preprint arXiv:2307.09782*, 2023.

[63] Xiaoxia Wu, Zhewei Yao, Minjia Zhang, Conglong Li, and Yuxiong He. Extreme compression for pre-trained transformers made simple and efficient. *arXiv preprint arXiv:2206.01859*, 2022.

[64] Haojun Xia, Zhen Zheng, Yuchao Li, Donglin Zhuang, Zhongzhu Zhou, Xiafei Qiu, Yong Li, Wei Lin, and Shuaiwen Leon Song. Flash-llm: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity. *arXiv preprint arXiv:2309.10285*, 2023.

[65] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. Sheared llama: Accelerating language model pre-training via structured pruning. *arXiv preprint arXiv:2310.06694*, 2023.

[66] Guangxuan Xiao, Ji Lin, Mickael Seznec, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022.

[67] Zhewei Yao, Reza Yazdani Aminabadi, Stephen Youn, Xiaoxia Wu, Elton Zheng, and Yuxiong He. Zeroquant-hero: Hardware-enhanced robust optimized post-training quantization framework for w8a8 transformers. *arXiv preprint arXiv:2310.17723*, 2023.

[68] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *arXiv preprint arXiv:2206.01861*, 2022.

[69] Zhewei Yao, Xiaoxia Wu, Cheng Li, Stephen Youn, and Yuxiong He. Zeroquant-v2: Exploring post-training quantization in llms from comprehensive study to low rank compensation. *arXiv preprint arXiv:2303.08302*, 2023.

[70] Zhihang Yuan, Lin Niu, Jiawei Liu, Wenyu Liu, Xinggang Wang, Yuzhang Shang, Guangyu Sun, Qiang Wu, Jiaxiang Wu, and Bingzhe Wu. Rptq: Reorder-based post-training quantization for large language models. *arXiv preprint arXiv:2304.01089*, 2023.

[71] Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. Q8BERT: Quantized 8bit bert. *arXiv preprint arXiv:1910.06188*, 2019.

[72] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

[73] Tianyi Zhang, Zhiqiu Lin, Guandao Yang, and Christopher De Sa. Qpytorch: A low-precision arithmetic simulation framework, 2019.

[74] Yijia Zhang, Lingran Zhao, Shijie Cao, Wenqiang Wang, Ting Cao, Fan Yang, Mao Yang, Shanghang Zhang, and Ningyi Xu. Integer or floating point? new outlooks for low-bit quantization on large language models. *arXiv preprint arXiv:2305.12356*, 2023.

[75] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

[76] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x. In *KDD*, 2023.

A Background of Quantization

Throughout this work, we focus on post-training quantization (PTQ), i.e., no or minimal training effort is applied after quantization, for which large accuracy degradation usually exhibits for coarse-grained quantization (per matrix/tensor) due to their large quantization error. Particularly, we use the per-row quantization (one row of the weight matrix) from [68] as our coarsest-grained quantization method, and we use block-k quantization (for every k elements, they have their own scaling factor and/or zero point) as our finer-grained quantization scheme.

Table A.1: GPT4-evaluation for the same model with different precision [75]. There is no clear relation between different bits and rating performance.

Model (Precision)	Writing	Roleplay	Coding	STEM	Humanities	Reasoning	Math	Extraction
vicuna-7b-v1.5 (Baseline)	7.55	7.95	3.21	8.36	9.68	4.6	3.6	6.4
vicuna-7b-v1.5 (INT4)	7.37	7.60	3.21	8.61	9.34	5.7	2.6	6.2
vicuna-7b-v1.5 (FP6)	7.55	7.88	3.67	8.55	9.21	4.8	2.2	6.1
vicuna-13b-v1.5 (Baseline)	7.90	7.75	2.73	7.32	9.24	5.2	2.2	6.0
vicuna-13b-v1.5 (INT4)	8.42	7.56	2.91	7.93	9.58	4.7	2.7	6.4
vicuna-13b-v1.5 (FP6)	8.06	7.60	2.94	7.76	9.31	4.8	2.8	5.9

Table A.2: **Zero-Shot Evaluation (Accuracy)**. Compare between GPTQ$_{C4}$ and RTN quantization algorithms for INT4 weight (W4A16) on LLaMA-1B (Top) and LLaMA-13B (Bottom). We apply fine-grain quantization (FGQ) in which the block-size is 256 elements per scale except for LLaMA-1B's (which is 128). arcC (arcE) stands for arc_challenges (arc_easy).

Models	Precision	FGQ	arcC	arcE	boolq	cb	copa	piqa	rte	wic	wsc	storycloze	MEAN
LLaMA-1B (4096-seq)	FP16	N/A	26.71	53.11	61.13	39.29	76.00	73.83	50.18	50.00	36.54	69.64	53.64
	INT4-GPTQ	✓	26.37	50.59	61.59	46.43	79.00	73.34	48.01	50.00	36.54	68.24	54.01
	INT4-RTN	✓	26.11	51.09	58.07	50.00	74.00	72.91	48.38	50.00	36.54	68.36	53.55
	FP5-GPTQ	✗	25.51	51.85	61.22	32.14	80.00	73.01	48.38	50.00	36.54	68.94	52.76
	FP5-RTN	✗	26.62	50.67	61.41	39.29	79.00	72.85	50.18	50.00	36.54	69.19	53.58
	FP6-GPTQ	✗	26.37	52.53	61.19	42.86	75.00	73.5	51.26	50.00	36.54	69.76	53.90
	FP6-RTN	✗	26.37	52.95	60.95	37.50	78.00	73.39	54.51	50.00	36.54	69.64	53.99
LLaMA-13B (2048-seq)	FP16	N/A	43.86	74.58	68.53	50.00	90.00	79.00	65.34	50.00	35.58	78.23	63.51
	INT4-GPTQ	✓	43.00	73.44	67.83	41.07	93.00	78.78	62.45	50.16	36.54	78.17	62.44
	INT4-RTN	✓	44.03	74.45	67.37	44.64	91.00	78.84	63.18	49.84	36.54	78.42	62.83
	FP5-GPTQ	✗	42.92	73.70	65.81	44.64	90.00	78.67	64.62	50.00	36.54	78.23	62.51
	FP5-RTN	✗	41.72	74.03	68.47	39.29	90.00	78.56	62.45	50.31	36.54	78.61	62.00
	FP6-GPTQ	✗	43.69	73.99	67.28	48.21	90.00	78.84	64.98	50.31	36.54	78.42	63.23
	FP6-RTN	✗	43.77	74.20	68.38	46.43	91.00	78.84	65.34	49.84	36.54	78.23	63.26
LLaMA-65B (2048-seq)	FP16	N/A	47.01	75.08	82.32	64.29	91.00	81.61	71.48	58.31	60.58	79.57	71.13
	INT4-GPTQ	✓	46.84	75.08	80.76	58.93	94.00	81.18	72.92	56.27	60.58	79.31	70.59
	INT4-RTN	✓	47.10	75.25	81.47	62.50	95.00	81.23	69.68	57.21	62.50	79.63	71.16
	FP5-GPTQ	✗	46.50	75.51	82.35	69.64	93.00	81.28	71.84	57.05	57.69	79.76	71.46
	FP5-RTN	✗	46.50	75.59	82.87	60.71	94.00	81.39	73.65	57.21	60.58	80.08	71.26
	FP6-GPTQ	✗	46.84	74.96	82.51	64.29	91.00	81.23	70.04	59.72	61.54	79.63	71.18
	FP6-RTN	✗	47.10	74.66	82.69	64.29	92.00	81.99	70.76	58.15	57.69	79.31	70.86

B Detailed Performance of GPU Kernels

The shapes of weight matrices are set according to the model specification of LLaMA-1B, LLaMA-13B, and LLaMA-65B, respectively. We mainly evaluate the kernel performance when the inference batch size is 8. As for the Fine-grained INT4 kernel, we set its quantization group size to 128 for the best of its performance. All the kernel latency shown here is measured in milliseconds.

Table B.1: The corresponding number for Figure 2.

Layer Name	Weight Shape	Input Shape	cuBLAS	Fine-grained INT4	FP6 (w/o Bias-Shift)	FP6 (w/ Bias-Shift)
FFN1-1b	5504*2048	2048*8	0.016	0.019	0.021	0.013
FFN2-1b	2048*5504	5504*8	0.02	0.043	0.022	0.016
FFN1-13b	13824*5120	5120*8	0.118	0.044	0.063	0.052
FFN2-13b	5120*13824	13824*8	0.109	0.106	0.068	0.053
FFN1-65b	22016*8192	8192*8	0.263	0.098	0.145	0.111
FFN2-65b	8192*22016	22016*8	0.266	0.167	0.157	0.114